안녕하세요.
Javascript 강좌중인 Falcon 입니다.

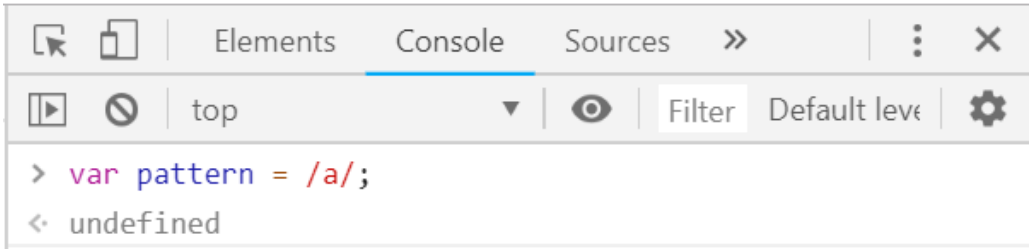
정규표현식이라는 것은 문자열에서 특정 문자를 찾아내는 도구입니다. 그런데 이 정규표현식이라는 것은 Javascript라는 언어 뿐만 아니라 다양한 언어에 정규표현식이라는 것이 존재 합니다. 다만 사용방법이 언어마다 조금씩 다를 뿐입니다. 그럼 코딩을 통해서 한 번 알아보도록 합시다.

블로그 : http://blog.naver.com/fb_lua 입니다.
네이버 : http://naver.com 입니다.

윗 부분을 보면 2개의 링크가 존재합니다. 아마 html을 배우신 분들은 이러한 결과 값을 출력하는 것이 어렵지는 않을 것 입니다. 그런데 만약에 이러한 링크가 100개 1000개를 입력해야하는 상황이 온다면?

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
  <head>
    <title>Javascript</title>
    <meta charset="utf-8">
  </head>
  <body>
    블로그 : <a href="http://blog.naver.com/fb_lua">http://blog.naver.com/fb_lua</a> 입니다. <br />
    네이버 : <a href="http://naver.com">http://naver.com</a> 입니다.
  </body>
</html>
```

이렇게 하나하나 등록하기에는 무리가 있습니다. 주소값을 하나하나 복사를 해야한다는 단점이 생기죠. 그럴 때 정규표현식을 사용하면 조금 더 편하게 이를 등록할 수 있습니다. 물론 요즘은 이런형식으로 등록을 하지 않겠지만 예제라고 생각하고 한번 봐주시길 봐주시길 바랍니다. 그전에 정규표현식의 사용법을 간단하게나마 코딩으로 알아보면서 같이 배워봅시다.




var pattern은 변수 입니다. 여기서 중요한것은 변수 안에 담긴 값입니다. /a/라는 값이죠.
이 값은 /슬러쉬와 슬러쉬 안의 값을 찾겠다라는 의미로 해석하시면 됩니다.

```
> var pattern = new RegExp('a');
<· undefined
```

그리고 위와 같은 뜻으로 사용되는 또 다른 방법이지만 이 두가지의 차이점은 존재한다고 합니다. 그 차이점에 대해서는 알게되면 여러분들에게 알려드리도록하고 두가지의 방법이 있다는 사실만 알고 넘어가시면 될 거 같습니다. 위를 통해서 여러분들이 찾을 문자를 선택을 했다면 이를 실행해서 찾아봅시다.

```
> var pattern = /a/;
<· undefined
> console.log(pattern.exec('abcdef'));
                                                VM562:1
  ▶ ["a", index: 0, input: "abcdef", groups: undefined]
```

console.log는 콘솔창에 출력을 해주는 기능이라는 것을 여러분들은 이미 알고 계실 것 입니다. 여기서 pattern은 무엇일까? pattern은 우리가 위에서 만들어놓은 패턴이다. a라는 문자열을 찾기 위해서 선언해둔 것이고 exec는 그 패턴을 실행을 하겠다라는 뜻이다. ()괄호 안을 자세히 보면 abcdef라는 문자열이 담겨 있음을 알 수 있다. 이는 이 괄호안에 있는 문자열중에서 a라는 문자열이 있는지 확인해라 결과값은 보면 알 수 있듯이 a라는 문자열을 찾았다고 알려주고 있다.

```
> var pattern = /a./;
<· undefined
> console.log(pattern.exec('abcdef'));
                                                VM6172:1
  ▶ ["ab", index: 0, input: "abcdef", groups: undefined
    ]
<· undefined
```

이번에는 찾고자하는 패턴이 바뀌었다는 것을 알 수 있다. /a./ 정규표현식에서 .은 조금 특별한 의미를 지니고 있다. 먼저 우리가 찾고자하는 문자는 a라는 것을 알 수 있지만 뒤의 .의 의미는 하나의 문자라는 의미이다. 즉, a뒤에 존재하는 어떠한 문자라도 하나의 문자를 더 찾아주게 된다. 그래서 이를 출력해보면 a와 a뒤에있던 b라는 문자를 찾아 ab가 출력됨을 볼 수 있다.

```
> var pattern = /a/;
<· undefined
> console.log(pattern.exec('bcdef'));
  null                                    VM6209:1
```

만약 찾고자 하는 문자열이 없다면 결과값은 무엇이 뜰지 궁금하실텐데 결과값은 null이 뜨게 된다.
이렇게 문자를 찾아내주는 exec가 있다면 boolean 형식으로 알려주는 test가 존재한다.

```
> var pattern = /a/;
<· undefined
> console.log(pattern.test('abcdef'));
  true                                    VM6267:1
<· undefined
> console.log(pattern.test('bcdef'));
  false                                   VM6270:1
<· undefined
```

위를 보면 알 수 있듯이 exec가 test로 바꼈을 뿐이다. 출력결과를 보면 알 수 있듯 문자열안에 값이 존재 한다면 true 존재하지 않는다면 false를 출력하는 것을 볼 수 있다. 정규표현식에서 문자열이 가지고 있는 메소드들중에서 사용할 수 있는 메소드들이 있는데 2가지만 한번 살펴보자.

```
> var pattern = /a/;
<· undefined
> var str = 'abcdef';
<· undefined
> str.match(pattern);
<· ▶["a", index: 0, input: "abcdef", groups: undefined]
```

match 메소드는 위에서 배운 exec와 비슷하다. 차이점은 아래에서 설명하도록 하겠습니다.

```
> var pattern = /a/;
<· undefined
> var str = 'abcdef';
<· undefined
> str.replace(pattern, 'A');
<· "Abcdef"
```

두번째 메소드는 replace는 패턴을 검색해서 이를 변경한 후 값을 리턴한다. 즉, 소문자 a를 찾아서 대문자 A로 변경을 해주게 된다. 그래서 위의 결과값을보면 Abcdef라는 문자열이 출력된 것을 볼 수 있다.

```
console.log(str.replace(pattern, 'A'));
```

혹시나 오랜만이라서 까먹은 분들을 위해서 알려드립니다. 크롬의 콘솔은 console.log를 자동으로 입력을 해주기 때문에 저렇게해도 결과값을 출력한다는 사실을 잊지 마시길 바랍니다. 이번에는 정규표현식의 옵션에 대해서 알아보도록 합시다. 옵션도 2가지 정도 알려드리도록 하겠습니다.

```
> var pattern = /a/;
<· undefined
> console.log(pattern.exec('Abcdef'));
  null                                    VM6655:1
<· undefined
```

소문자 a를 찾으려고하는데 대문자 A가 있을 경우 보통의 방법으로는 이를 찾을 수 없습니다. 하지만 옵션을 사용하면 소문자 a로 검색을 해도 대문자 A를 찾을 수 있게 해줄 수 있습니다. 먼저 한가지 알아보도록 하죠.

```
> var pattern = /a/i;
<· undefined
> console.log(pattern.exec('Abcdef'));
                                          VM6672:1
  ▶["A", index: 0, input: "Abcdef", groups: undefined]
```

위를 보시면 찾는 문자열 소문자 /a/ 옆으로 i라는 문자를 주었습니다. 여기서 i는 대소문자를 가리지 않고 원하는 문자열을 찾겠다라는 의미입니다. 출력결과를 보시면 알 수 있듯이 대문자 A를 찾아낸 것을 볼 수 있습니다. 이처럼 옵션을 사용하면 좀 더 편하게 정규표현식을 사용할 수 있습니다.

```
> var pattern = /a/;
<· undefined
> console.log(pattern.exec('abcdefa'));
                                                VM6705:1
  ▸ ["a", index: 0, input: "abcdefa", groups: undefined
    ]
<· undefined
```

위를 보면 찾고자하는 문자열에 a가 2개 있습니다. 처음과 끝에 있지만 평범한 방법으로는 a라는 문자를 하나 밖에 찾지 못합니다. 여러분들이 원하는 문자를 두번 찾고 싶다면 g라는 옵션을 사용하시면 됩니다.

```
> var pattern = /a/g;
<· undefined
> console.log(pattern.exec('abcdefa'));
                                                VM6890:1
  ▸ ["a", index: 0, input: "abcdefa", groups: undefined
    ]
<· undefined
> console.log(pattern.exec('abcdefa'));
                                                VM6893:1
  ▸ ["a", index: 6, input: "abcdefa", groups: undefined
    ]
<· undefined
```

exec는 한번의 실행이기 때문에 a라는 문자열을 한번씩 출력합니다. index번호를 보시면 찾는 위치가 다르다는 것을 볼 수 있습니다. 위에서 match가 exec와 비슷하다고 설명을 해드렸습니다.

```
> var pattern = /a/g;
<· undefined
> console.log('abcdefa'.match(pattern));
  ▸ (2) ["a", "a"]                              VM6908:1
```

직접 해보시면 아시겠지만 match는 a라는 문자를 2번 찾았다는 것을 바로 출력을 해주게 됩니다. 이러한 차이점이 있을 뿐 크게 차이는 없을거 같습니다.  이렇게해서 정규표현식의 기초에 대해서 알아보았습니다.

이것으로 [javascript 강좌] 제 13강 정규표현식의 기초 편을
마치도록 하겠습니다.